Chapter 4
FOUNDATIONS OF PREDICTIVE MODELING IN DATA SCIENCE

Instructor: Dr. Heyfa Ammar

Slides Courtesy: Prof. Anis Koubaa

We iterate on a data set to get a model [training the model]

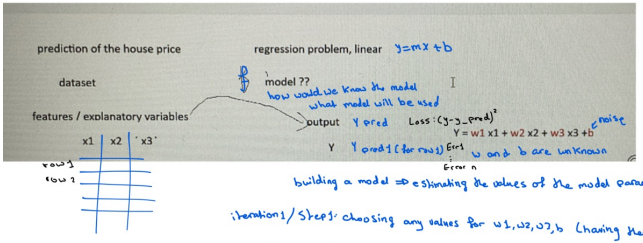
MSE = mean squared error

Loss function depends on w

Loss(w)

Objective: reduce the loss: get a Avg Loss 2 < Avg Loss 1

Avg Loss(n+1) < Avg Loss(n)



First find Y_pred for each row then find the error for each row

building a model ⇒ estimating the values of the model parameters ⇒ w1=?, w2=?, w3=?, b=?

iteration 1/ step 1: choosing any values for w1, w2, w3, b (having these values known means we have the model)

then we'll repeat the mode with a better estimate with less loss and we can repeat this multiple times till the error gets rly low

Optimizing a function: Minimize / Maximize the function. Building a model is an optimizing function because we want to minimize the error

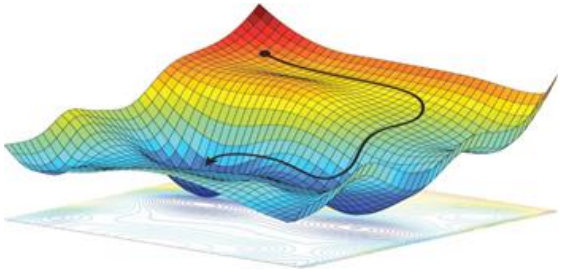
$f(x) = x^2 - 1$ ⇒ find the 1st derivative ⇒ $f'(x) = 0$ [function with 1 variable only] more than one variable, we use partial derivative with respect to one variable and the other variables will be constant

we need to get the gradient for many variables, we have the values for x and we need to adjust w where we need to minimize the loss function

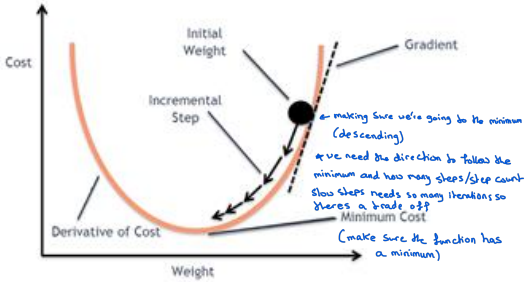
$\frac{df}{dx1}$; $\frac{df}{dx2}$; $\frac{df}{dx3}$ ⇒ x1, x2, x3 Loss (w1, w2, w3, b) = ... (minimize it)

Optimization in Data Science

- **Objective:** Identify the optimal model within a predefined class of mathematical models.
  - **Minimize the error between predicted values and actual observations**
- **Methodology:** Employ efficient search or optimization techniques for model fitting.
  - Example: Gradient Descent
- **Application Example:**
  - Maximizing **recommendation system** effectiveness by tuning algorithms.
  - Enhancing **weather forecast** precision by calibrating prediction models.
  - Improving **email spam detection** accuracy by refining logistic regression parameters.



https://medium.com/analytics-vidhya



https://www.analyticsvidhya.com/

STATISTICAL LEARNING

Convex optimization

CS316: INTRODUCTION
TO DATA SCIENCE and AI

What is Optimization?

- **Definition of Optimization**
  - Optimization involves **selecting the best element** from a set of available alternatives.
  - In mathematical terms, this process is often associated with **finding the minimum or maximum** of a function.

Importance in Data Science

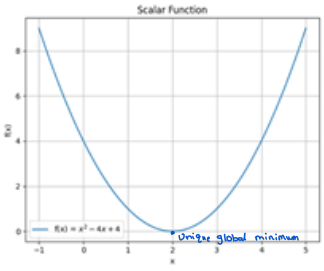
- **Core Component**
  - Optimization is the **backbone of machine learning**.
  - Enables models to **learn from data** by systematically **improving performance** according to specified **metrics (i.e., loss function)**.
- **Objective Function**
  - **Machine learning** models are trained by **minimizing** or **maximizing** an **objective function**, also known as a **loss** or **cost** function.
  - This function measures the **error** or the discrepancy between the ***predicted*** values and the ***actual*** values in the training data.

define the objective of what we want to maximize/minimize
defining the error

Convex Optimization Problems in Data Science

- **Linear Regression**
  - **Problem:** **Minimize** the **sum of squared residuals** (MSE) between observed values and values predicted by a linear model.
  - **Nature:** make sure it has a min **Convex problem** as the objective function is a **quadratic function**, ensuring a single global minimum.
- **Logistic Regression:**
  - **Problem: Maximize** the **likelihood** of correctly predicting binary outcomes using a logistic function.
  - **Nature:** Convex problem due to the **log-likelihood** function being concave; minimization of its negative is a convex optimization problem.

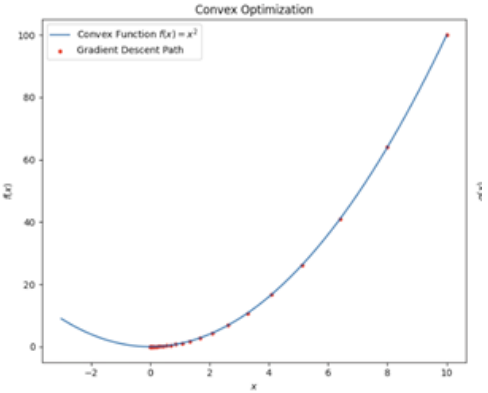
$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

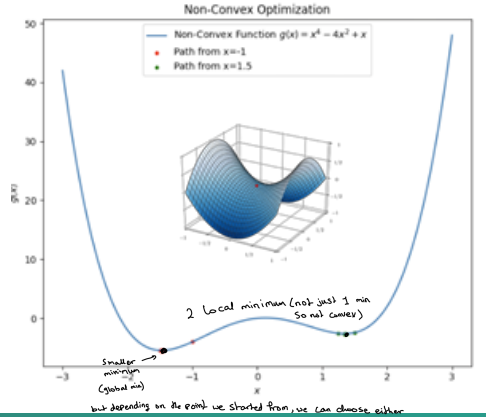


$$L(y, \hat{p}) = -\left[y \log(\hat{p}) + (1-y)\log(1-\hat{p})\right]$$

Types of Optimization Problems

- **CONVEX OPTIMIZATION PROBLEMS**
  - **Definition:** An optimization problem where the **objective function** is a **convex function** and the feasible set is a convex set.
  - **Characteristics:** Unique **global minimum**; any **local minimum** is also a global minimum, simplifying the search for solutions.
  - **Example:** Least squares linear regression, where the function to minimize is a quadratic function of the parameters.
- **NON-CONVEX OPTIMIZATION PROBLEMS**
  - **Definition:** An optimization problem where the **objective function** or the feasible set is **non-convex**.
  - **Characteristics:** Potential for multiple **local minima** and possibly saddle points, making these problems more challenging to solve.
  - **Example: Neural network** training, where the **loss landscape** is **highly non-linear** and contains **many local minima**.





MATH REFRESHER

CS316: INTRODUCTION TO DATA SCIENCE and AI

Mathematical Notations in Convex Optimization

Overview

- **Purpose**: Understand the fundamental notations used in convex optimization to describe functions, derivatives, and optimization conditions.
- **Importance**: Mastery of these notations is crucial for formulating and solving optimization problems effectively.

Key Components

- **Scalar Functions**: Represented as $f(x)$ where x is a scalar.
- **Vector Functions**: Denoted by $f(\mathbf{x})$ where $\mathbf{x}$ is a vector of variables.

Scalar and Vector Functions

Scalar Functions

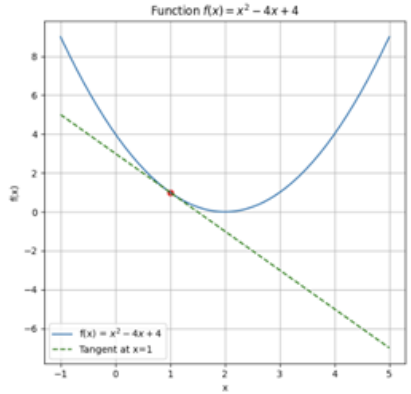
- **Function Notation**: $f(x)$ — A function with a single variable.
- **Derivative**:
  - $f'(x)$: Derivative of f, showing the rate of change at point x.

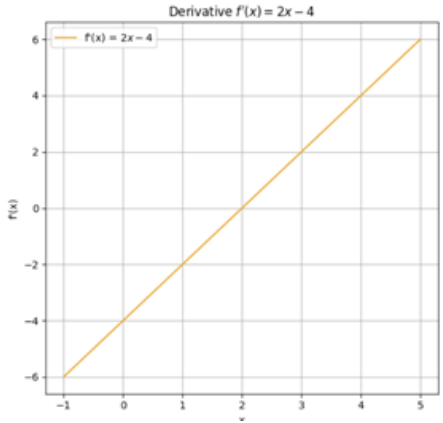
Scalar Function

For the scalar function, we'll use:

$$f(x) = x^2 - 4x + 4$$

We'll compute its derivative and visualize both the function and its derivative.





Scalar and Vector Functions

The size of the gradient vectors in a field plot typically represents the magnitude of the gradient at that point.

Vector Functions

- **Function Notation**: $f(\mathbf{x})$ — A function with multiple variables.
- **Gradient**:
  - $\nabla f(\mathbf{x})$: A vector of partial derivatives, representing the steepest ascent direction of f.
  - Example:

Gradient will give us the opposite sign of the min point
↓
want be asked to calculate it in exam

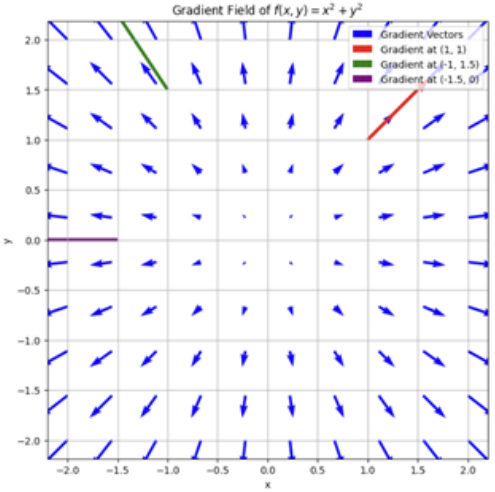
$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$$

Vector Function

For the vector function, we'll consider a function of two variables:

$f(x, y) = x^2 + y^2$

We'll compute and visualize the gradient vector field of this function.




```
The gradient at (1, 1) is: [0.70710678 0.70710678]
The gradient at (-1, 1.5) is: [-0.70710678  1.06066017]
The gradient at (-1.5, 0) is: [-1.06066017  0.        ]
```

Red Arrow at (1, 1): This arrow shows the direction and magnitude of the gradient at $(1, 1)$. Since $f(x, y) = x^2 + y^2$ is a paraboloid centered at the origin, the gradient at $(1, 1)$ points directly away from the origin. The length of the arrow indicates that the function increases rapidly in that direction.

Vector Functions

For a 2D vector $\mathbf{v} = [v_x, v_y]$, the magnitude (or length) of the vector is given by the Euclidean norm:

$$|\mathbf{v}| = \sqrt{v_x^2 + v_y^2}$$

The angle θ of the vector with respect to the positive x-axis is given by the arctangent of the ratio of its y-component to its x-component:

$$\theta = \arctan\left(\frac{v_y}{v_x}\right)$$

Given your vector $\mathbf{v} = [0.70710678, 0.70710678]$:

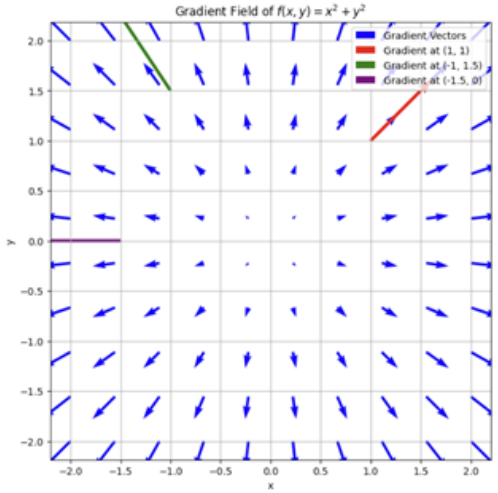
1. **Magnitude**:
   The magnitude can be calculated as:

$$|\mathbf{v}| = \sqrt{0.70710678^2 + 0.70710678^2}$$

2. **Angle**:
   Assuming the angle is measured from the positive x-axis in a counter-clockwise direction (standard in mathematics), the angle can be calculated using the `arctan` (or `atan2` which is more numerically stable and handles division by zero):

$$\theta = \arctan\left(\frac{0.70710678}{0.70710678}\right) = \arctan(1)$$

The size of the gradient vectors in a field plot typically represents the magnitude of the gradient at that point.



Red Arrow at (1, 1): This arrow shows the direction and magnitude of the gradient at $(1, 1)$. Since $f(x, y) = x^2 + y^2$ is a paraboloid centered at the origin, the gradient at $(1, 1)$ points directly away from the origin. The length of the arrow indicates that the function increases rapidly in that direction.

The Jacobian Matrix | *First-Order Derivative*

What is the Jacobian Matrix?

- **Definition**: The Jacobian matrix is a matrix of all first-order partial derivatives of a vector-valued function. The gradient is needed to know the min
- **Mathematical Representation**:
  - For a function $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^m$ that maps n inputs to m outputs, the Jacobian matrix J of $\mathbf{f}$ is given by:

$$J(\mathbf{f})(\mathbf{x}) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

- **Purpose**: The Jacobian matrix is crucial for understanding how changes in the input variables affect the outputs, and it generalizes the concept of derivatives to multidimensional functions.

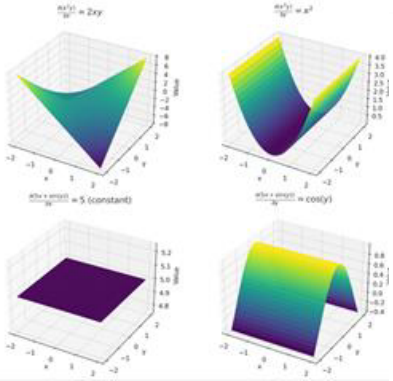
```python
import sympy as sp

# Define the variables
x, y = sp.symbols('x y')

# Define the function
f = sp.Matrix([x**2 * y, 5*x + sp.sin(y)])

# Compute the Jacobian
J = f.jacobian([x, y])
J
```

$$\begin{bmatrix} 2xy & x^2 \\ 5 & \cos(y) \end{bmatrix}$$



The Jacobian Matrix in Optimization

Importance in Optimization

- **Sensitivity Analysis**: The Jacobian matrix helps determine how sensitive the output variables are to changes in the input variables.
- **Multivariable Functions**: Essential for problems where multiple dependent variables need optimization, the Jacobian provides a comprehensive view of the gradient across all dimensions.
- **Use in Numerical Methods**: The Jacobian is particularly important in numerical optimization techniques like Newton's method for systems of equations, where it is used to find zeros or solve optimization problems efficiently.

The Hessian Matrix | *Second-Order Derivative*

Matrix Functions

- **Hessian Matrix:**
  - $H(f)(\mathbf{x})$: A square matrix of second-order partial derivatives. It describes the local curvature of f. to check for global min (is it convex or not?
  - Representation:

+ve → has a global min (convex)

-ve → doesn't have a min (not convex)

$$H(f)(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

Importance of the Hessian

- **Convexity Check**: f is convex if $H(f)(\mathbf{x})$ is positive semidefinite across the domain of f.

1. **Semidefinite Matrices**: In linear algebra and optimization, a matrix is said to be positive semidefinite if it satisfies the condition $x^T Ax \geq 0$ for all vectors x. This concept is central to various optimization problems and matrix theory.

Consider the function $f(x, y) = x^2 + xy + y^2$. We will:

1. Compute its Hessian matrix.
2. Evaluate the Hessian matrix at a specific point.
3. Discuss the significance of the Hessian matrix in terms of curvature and optimization.

```python
import sympy as sp

# Define the variables
x, y = sp.symbols('x y')

# Define the function
f = x**2 + x*y + y**2

# Compute the second partial derivatives
f_xx = sp.diff(f, x, x)
f_xy = sp.diff(f, x, y)
f_yx = sp.diff(f, y, x)
f_yy = sp.diff(f, y, y)

# Form the Hessian matrix
Hessian = sp.Matrix([[f_xx, f_xy],
                     [f_yx, f_yy]])

# Display the Hessian matrix
print("Hessian Matrix of the function:")
sp.pprint(Hessian)

# Evaluate the Hessian matrix at a specific point, for example at (x, y) = (1, 1)
Hessian_at_point = Hessian.subs({x: 1, y: 1})
print("\nHessian Matrix at (x, y) = (1,1):")
sp.pprint(Hessian_at_point)

# Compute eigenvalues to determine the nature of critical points
eigenvals = Hessian_at_point.eigenvals()
print("\nEigenvalues of the Hessian matrix at (x, y) = (1,1):")
sp.pprint(eigenvals)
```

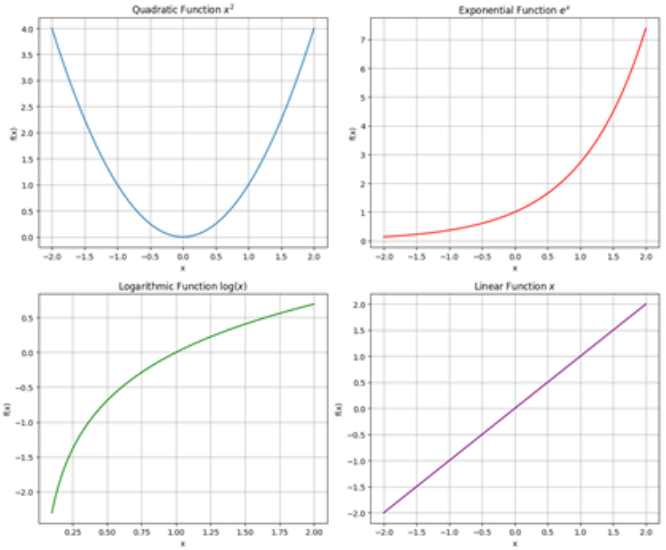
```
Hessian Matrix of the function:
⎡2  1⎤
⎢    ⎥
⎣1  2⎦

Hessian Matrix at (x, y) = (1,1):
⎡2  1⎤
⎢    ⎥
⎣1  2⎦

Eigenvalues of the Hessian matrix at (x, y) = (1,1):
{1: 1, 3: 1}
```

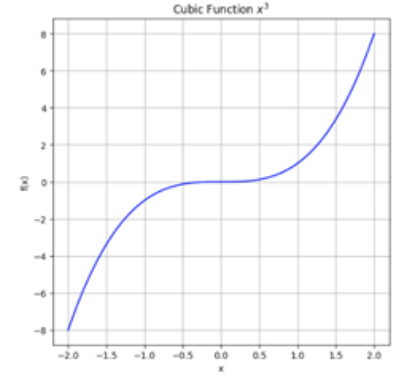
Examples of Convex Functions

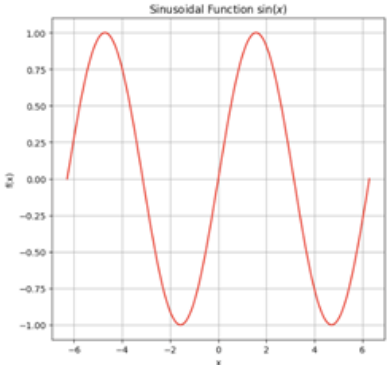
- **Quadratic Functions**:
  - Example: $f(x) = x^2$, which is convex because the second derivative $f''(x) = 2$ is always positive.
- **Exponential Functions**:
  - Example: $f(x) = e^x$, with its second derivative $f''(x) = e^x$ also being positive.
- **Logarithmic and Linear Functions**:
  - Example: $f(x) = \log(x)$ (convex over $x > 0$).
  - Linear Function Example: $f(x) = ax + b$ is convex and concave (it is linear).



Examples of Non-Convex Functions

1. **Cubic Function**: $f(x) = x^3$ $f''(x) = 6x$ we can say much abt it since it has a variable
   - This function has a point of inflection at $x = 0$, which means it changes curvature from concave to convex, making it non-convex as a whole.
2. **Sinusoidal Function**: $f(x) = \sin(x)$
   - A sinusoidal function oscillates between positive and negative values, with its peaks and troughs making it clearly non-convex, as the line segments connecting points across a peak or trough will lie below the curve.
3. **Absolute Value Function**: $f(x) = |x|$
   - Although it might appear linear and convex at first glance, the absolute value function has a sharp point at $x = 0$, which violates the smoothness condition required for convex functions. It is technically neither convex nor concave due to this cusp.
4. **Polynomial Function with Multiple Roots**: $f(x) = x^4 - x^2$
   - This function, due to its multiple turning points, exhibits both concave and convex intervals, making it non-convex.
5. **Exponential Minus Quadratic**: $f(x) = e^{-x} - x^2$
   - This function has both exponential decay and quadratic growth components, creating multiple inflection points and thus making it non-convex.





LECTURE

CONVEX OPTIMIZATION

Optimization techniques for convex functions

CS316: INTRODUCTION TO DATA SCIENCE

Unconstrained vs. Constrained Optimization

Definitions & Key Features:

- **Unconstrained Optimization**:
  - **No Limits**: Optimizes a function $f(x)$ anywhere within its domain.
  - **Methods**: Uses simpler methods like Gradient Descent.
- **Constrained Optimization**:
  - **With Conditions**: Must satisfy additional constraints like $g(x) \leq 0$.
  - **Methods**: Requires complex techniques such as Lagrangian Multipliers or KKT Conditions.

```python
# Function to find the minimum using a simple gradient descent approach
def find_minimum_unconstrained(x_start, learning_rate, iterations):
    x = x_start
    for i in range(iterations):
        x -= learning_rate * df(x)
    return x

# Function to find the minimum considering the constraint x <= 1
def find_minimum_constrained(x_start, learning_rate, iterations):
    x = x_start
    for i in range(iterations):
        x_new = x - learning_rate * df(x)
        if g(x_new) <= 0:  # Check if the new x satisfies the constraint
            x = x_new
        else:
            break  # Stop if the constraint is violated
    return x

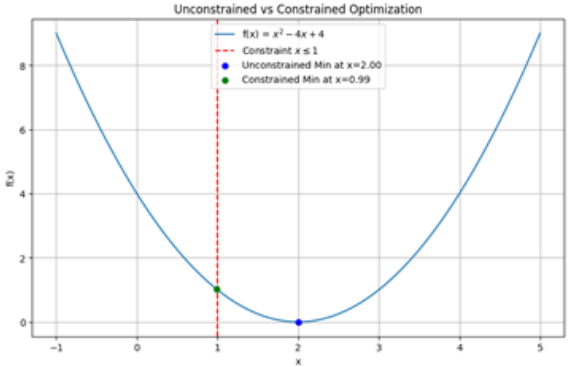
```

EXAMPLE

Function and Constraints

We'll use the function $f(x) = x^2 - 4x + 4$ for both cases:

1. **Unconstrained Optimization**: We'll find the minimum of the function over its entire domain.
2. **Constrained Optimization**: We'll add a constraint $g(x) = x - 1 \leq 0$, which means we're only allowed to find the minimum where $x \leq 1$.



Unconstrained vs. Constrained Optimization

Contrast:

- **Freedom**: Unconstrained has complete freedom in variable choices; Constrained is limited by specific rules.
- **Solution Space**: Unconstrained searches the entire domain; Constrained focuses on the feasible set.

Practical Application:

- **Unconstrained**: Parameter optimization in algorithms.
- **Constrained**: Resource allocation within budget limits.

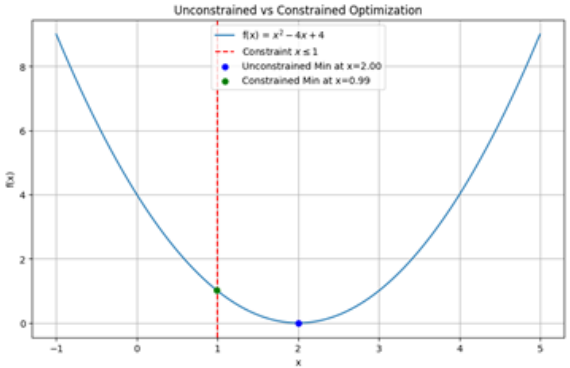
Conclusion:

- The choice between them depends on the problem constraints and the desired outcomes.

```python
# Function to find the minimum using a simple gradient descent approach
def find_minimum_unconstrained(x_start, learning_rate, iterations):
    x = x_start
    for i in range(iterations):
        x -= learning_rate * df(x)
    return x

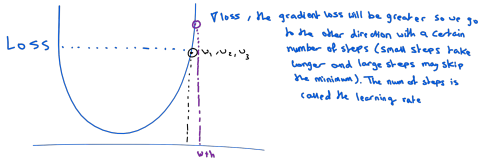
# Function to find the minimum considering the constraint x <= 1
def find_minimum_constrained(x_start, learning_rate, iterations):
    x = x_start
    for i in range(iterations):
        x_new = x - learning_rate * df(x)
        if g(x_new) <= 0:  # Check if the new x satisfies the constraint
            x = x_new
        else:
            break  # Stop if the constraint is violated
    return x

```



Unconstrained Optimization Techniques

Both techniques have pros and cons



1. **Gradient Descent:**
   - **Mathematical Concept**: Update formula:

     steps (chosen manually)

     $$x_{\text{new}} = x_{\text{old}} - \alpha \nabla f(x_{\text{old}})$$

     decrease the loss by going to the minimum

     gradient of the loss

     Here, α is the step size, and $\nabla f(x)$ is the gradient or slope of the function at x.
   - **Simple Explanation**: Like walking downhill, this method takes steps proportional to the steepness of the hill to reach the lowest point. The steeper the hill, the bigger the step.
2. **Newton's Method:** the α is not chosen randomly, it's chosen mathmatically (more computations as we need more calculations rather than a constant number)
   - **Mathematical Concept**: Uses the gradient and the Hessian (a matrix of second derivatives) for faster convergence:

     α is replaced by the inverse of the gradient matrix

     $$x_{\text{new}} = x_{\text{old}} - \left[\nabla^2 f(x_{\text{old}})\right]^{-1} \nabla f(x_{\text{old}})$$

   - **Simple Explanation**: Newton's method adjusts not just according to the slope but also considers how quickly the slope changes, allowing for quicker and more accurate steps towards the minimum.

Gradient Descent vs. Newton's Method

1. Simplicity and Ease of Implementation

- **Gradient Descent**:
  - Requires only first derivatives (gradients).
  - Simple update rule: $x_{\text{new}} = x_{\text{old}} - \alpha \nabla f(x_{\text{old}})$.
- **Newton's Method**:
  - Requires both first and second derivatives (Hessian matrix).
  - More complex update rule involving matrix inversion: $x_{\text{new}} = x_{\text{old}} - H^{-1} \nabla f(x_{\text{old}})$.
  - Implementing Newton's method can be more error-prone and computationally intensive because of the need to calculate and invert the Hessian matrix.

Gradient Descent vs. Newton's Method

2. Computational Efficiency

- **Gradient Descent**:
  - Less computationally intensive per iteration.
  - Scales better with the number of variables, making it suitable for high-dimensional problems.
- **Newton's Method**:
  - Each iteration is more computationally costly due to the need for computing and inverting the Hessian matrix.
  - Not well-suited for very large-scale problems where Hessian computation and inversion become infeasible.

Gradient Descent vs. Newton's Method

3. Robustness and Convergence

- **Gradient Descent**:
  - More robust to the initial choice of parameters.
  - Although potentially slower, it provides steady progress towards the minimum.
- **Newton's Method**:
  - Faster convergence near the minimum if the starting point is close to the minimum and the function is well-behaved.
  - Can exhibit poor performance or divergence if the Hessian is not positive definite or near-singular.

4. Applicability to Non-Convex Problems

- **Gradient Descent**:
  - Can be applied to both convex and non-convex problems, although it may only find a local minimum in non-convex scenarios.
- **Newton's Method**:
  - Best suited for convex problems where the objective function has a well-defined bowl shape. In non-convex problems, Newton's method can converge to saddle points or diverge.

Gradient Descent vs. Newton's Method

5. Tunability

- **Gradient Descent**:
  - Provides the flexibility to adjust the learning rate (α) to control convergence behavior.
- **Newton's Method**:
  - Less flexibility in tuning since the step size is implicitly determined by the Hessian.

LECTURE

CONVEX OPTIMIZATION

Gradient Descent

CS316: INTRODUCTION TO DATA SCIENCE and AI

Intuition behind gradient descent

What is Gradient Descent?

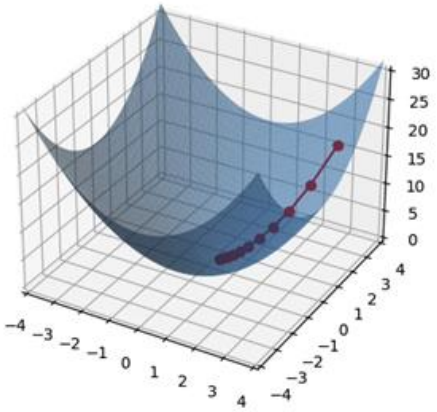
- **Gradient Descent** is an optimization algorithm used to minimize a function by iteratively moving towards the minimum value of the function.

How Does It Work?

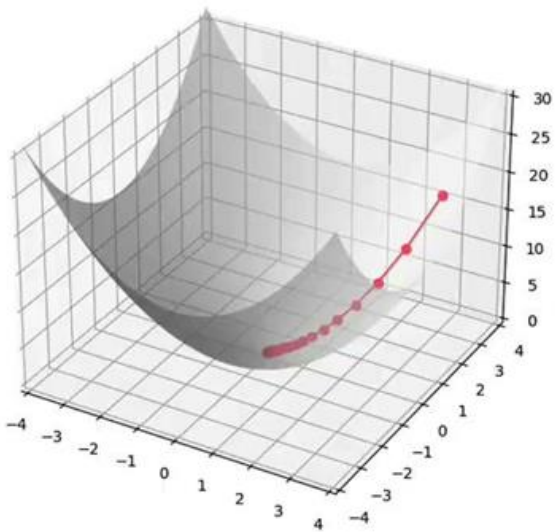
- **Step-by-Step Process**:
  1. **Start** with an initial guess for the value of the parameter(s).
  2. **Calculate the Gradient**: Determine the gradient (the slope of the function) at the current point.
  3. **Update the Parameter(s)**: Adjust the parameter(s) in the direction opposite to the gradient to move towards the minimum.

$$x_{\text{new}} = x_{\text{old}} - \alpha \nabla f(x_{\text{old}})$$

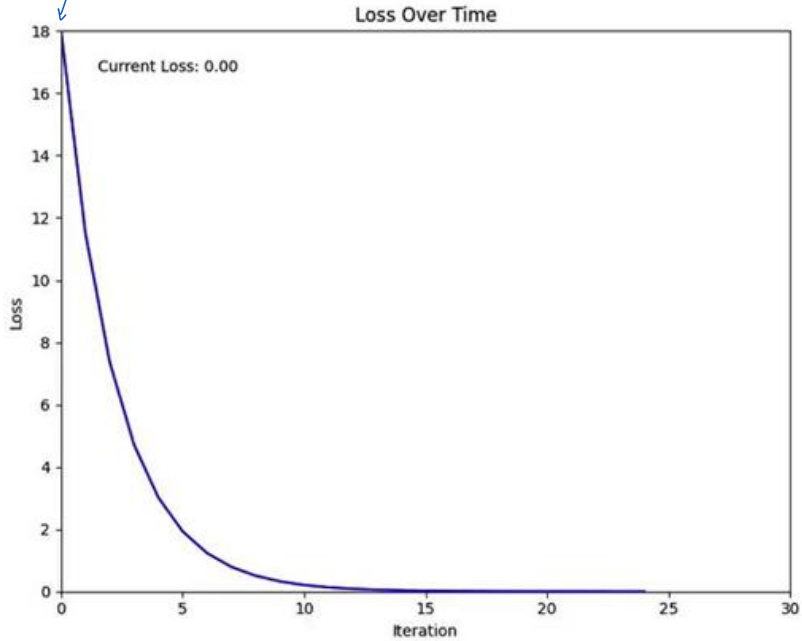
Where α is the learning rate, controlling the step size.



Unconstrained vs. Constrained Optimization



Initially loss is high then we try to minimize it by multiple iterations till the loss gets as close to 0



Mathematical Foundation of Gradient Descent

Objective:

- To minimize a function $f(x)$, where x can be a vector of parameters.

Derivation of the Update Rule:

1. **Taylor Expansion**:
   - To understand how the function f changes, we consider the Taylor expansion around a point x:

$$f(x + \Delta x) \approx f(x) + \nabla f(x)^T \Delta x + \frac{1}{2} \Delta x^T H \Delta x$$

   Here, $\nabla f(x)$ is the gradient of f at x, and H is the Hessian matrix of second derivatives.
2. **Neglect Higher-Order Terms**:
   - For small changes Δx, the higher-order terms (like the Hessian term) become negligible, simplifying to:

$$f(x + \Delta x) \approx f(x) + \nabla f(x)^T \Delta x$$

Mathematical Foundation of Gradient Descent

3. **Descent Direction**:
   - To decrease f, we choose Δx such that $f(x + \Delta x)$ is less than $f(x)$. The most effective direction to decrease f is opposite to the gradient, $\nabla f(x)$:

     $$\Delta x = -\eta \nabla f(x)$$

     where η (eta) is a small positive scalar known as the learning rate.
4. **Update Rule**:
   - Substituting Δx in the simplified Taylor expansion:

     $$f(x - \eta \nabla f(x)) \approx f(x) - \eta \nabla f(x)^T \nabla f(x)$$

   - Since $\nabla f(x)^T \nabla f(x)$ is always non-negative (it's the square of the gradient norm), the function value decreases if η is chosen properly.
5. **Gradient Descent Formula**:
   - The update rule for x to minimize f becomes:

     $$x_{\text{new}} = x - \eta \nabla f(x)$$

   - Each iteration moves x in the direction that most reduces f.

Gradient Descent Algorithm

Algorithm 3 Gradient Descent

1: **Input:** Loss function f, gradient ∇f, initial weights w_{init}, learning rate α, tolerance tol, maximum iterations max_iters
2: **Output:** Optimized weights w
3: **Initialize:** $w \leftarrow w_{\text{init}}$
4: **Initialize:** iter $\leftarrow 0$
5: **Initialize:** converged $\leftarrow$ False {Begin the optimization process}
6: **while not** converged **and** iter $<$ max_iters **do**
7: gradient $\leftarrow \nabla f(w)$ {Compute the gradient of the loss function with respect to weights}
8: $w \leftarrow w - \alpha \times$ gradient {Adjust weights to minimize the loss, moving against the gradient}
9: **if** $\|$ Loss $\|$ $<$ tol **then**
{Check if the Loss is small enough to assume convergence}converged $\leftarrow$ True
10:11: **end if**
12: iter $\leftarrow$ iter $+ 1$ {Update iteration counter}
13: **end while**
14: **return** w {Return the optimized weights} $=0$

Model Evaluation

Model Evaluation in Linear Regression problems

CS316: INTRODUCTION TO DATA SCIENCE and AI

Model Evaluation in Linear Regression problems

- MSE, RMSE, MAE
- Overfitting/Underfitting
- Tradeoff Bias/Variance

MSE, RMSE, MAE

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

Haven't we seen this already? ;)

Root Mean Squared Error (RMSE) RMSE= $\sqrt{MSE}$

What about MAE?

Overfitting/Underfitting

Understanding Overfitting

- **Overfitting Defined:**
  Overfitting occurs when a model learns the training data too closely, capturing noise and outliers, which results in poor performance on unseen data.
- **Training Loss Indicator:**
  $\text{Training Loss} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$
  Approaches zero in overfitting scenarios.



Overfitting Consequences

- **Generalization Failure:**

  Overfitting results in a substantial gap between training accuracy and the accuracy on novel, unseen data.

- **Quantifying Generalization Risk:**

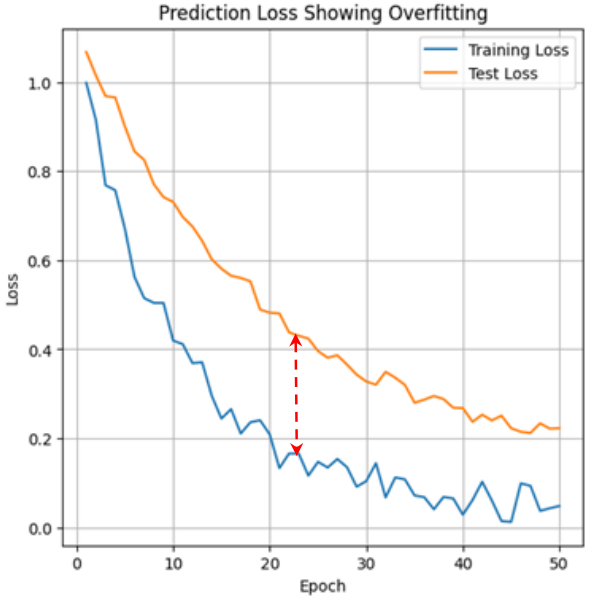
  $$\text{Generalization Risk} = \mathbb{E}[\text{Loss}(Y, g_{G\tau}(X))]$$

  $$l(g_{G\tau}) = \sum_{x,y} \text{Loss}(y, g_{G\tau}(x)) f(x, y)$$

  (In the continuous case, replace the sum with an integral.)

- **Impact of Overfitting on Generalization Risk:**

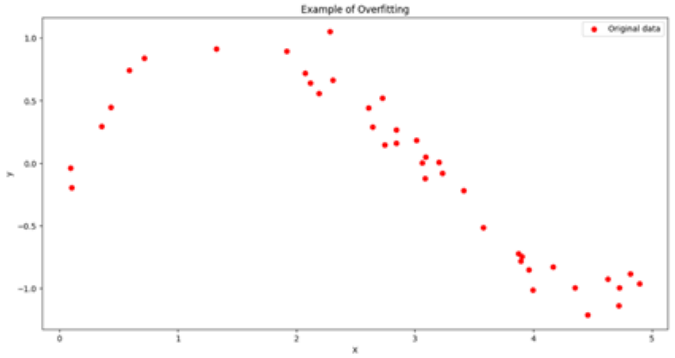
  Overfitting is marked by elevated generalization risk, signifying a model's poor generalization capability. This misalignment underscores the model's inability to accurately predict outcomes for unseen data, demonstrating the critical need for models that can generalize well beyond their training dataset.



Overfitting in Model Training

1. **Training Loss Minimization:**
   - Objective: Find g_{GT} within a function class G that minimizes training loss.
   - **Formula:**

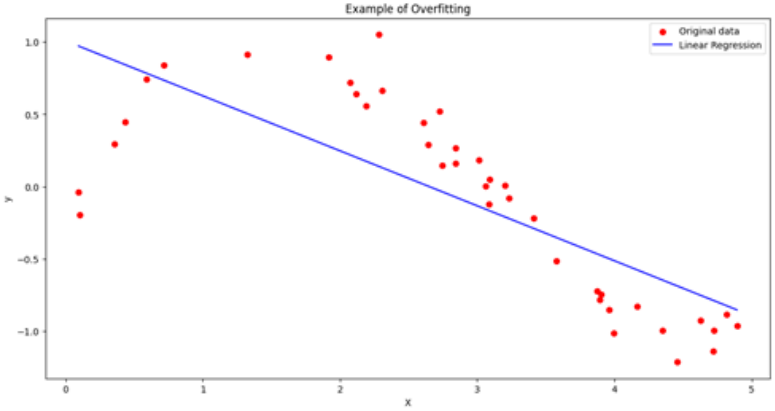
     $$g_{GT} = \arg\min_{g \in G} \frac{1}{n} \sum_{i=1}^{n} \text{Loss}(Y_i, g(X_i))$$
2. **Overfitting:**
   - Occurs when g_{GT} fits the training data too perfectly, leading to zero training loss.
   - **Implication:** Poor prediction on new, independent data due to lack of generalization.
3. **Example of Overfitting:**
   - A model where $g(X_i) = Y_i$ for all i in the training set, achieving zero squared-error training loss.



Overfitting in Model Training

1. **Training Loss Minimization:**
   - Objective: Find g_{GT} within a function class G that minimizes training loss.
   - **Formula:**

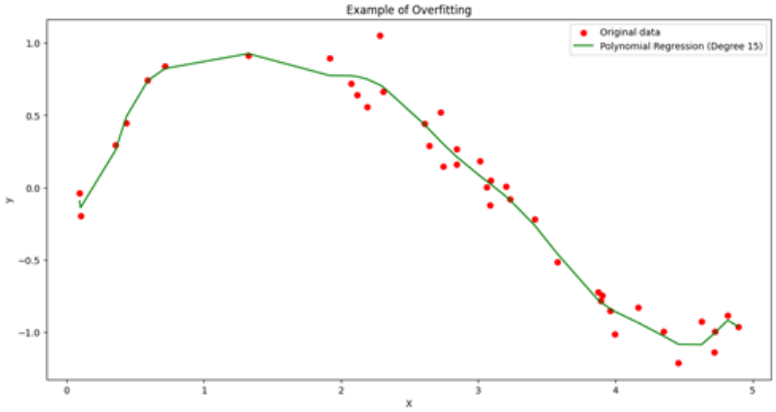
     $g_{GT} = \arg\min_{g \in G} \frac{1}{n} \sum_{i=1}^{n} \text{Loss}(Y_i, g(X_i))$
2. **Overfitting:**
   - Occurs when g_{GT} fits the training data too perfectly, leading to zero training loss.
   - **Implication:** Poor prediction on new, independent data due to lack of generalization.
3. **Example of Overfitting:**
   - A model where $g(X_i) = Y_i$ for all i in the training set, achieving zero squared-error training loss.



MSE = 0.1864

Overfitting in Model Training

1. **Training Loss Minimization:**
   - Objective: Find g_{GT} within a function class G that minimizes training loss.
   - **Formula:**
     $g_{GT} = \arg\min_{g \in G} \frac{1}{n} \sum_{i=1}^{n} \text{Loss}(Y_i, g(X_i))$
2. **Overfitting:**
   - Occurs when g_{GT} fits the training data too perfectly, leading to zero training loss.
   - **Implication:** Poor prediction on new, independent data due to lack of generalization.
3. **Example of Overfitting:**
   - A model where $g(X_i) = Y_i$ for all i in the training set, achieving zero squared-error training loss.

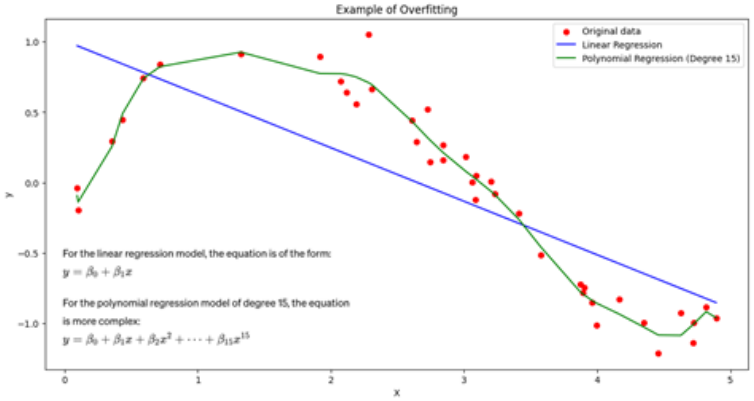


MSE = 0.010

Overfitting in Model Training

1. **Training Loss Minimization:**
   - Objective: Find g_{GT} within a function class G that minimizes training loss.
   - **Formula:**

     $$g_{GT} = \arg\min_{g \in G} \frac{1}{n} \sum_{i=1}^{n} \text{Loss}(Y_i, g(X_i))$$
2. **Overfitting:**
   - Occurs when g_{GT} fits the training data too perfectly, leading to zero training loss.
   - **Implication:** Poor prediction on new, independent data due to lack of generalization.
3. **Example of Overfitting:**
   - A model where $g(X_i) = Y_i$ for all i in the training set, achieving zero squared-error training loss.



Linear Regression Equation:

$$y = 1.01 - 0.38x$$

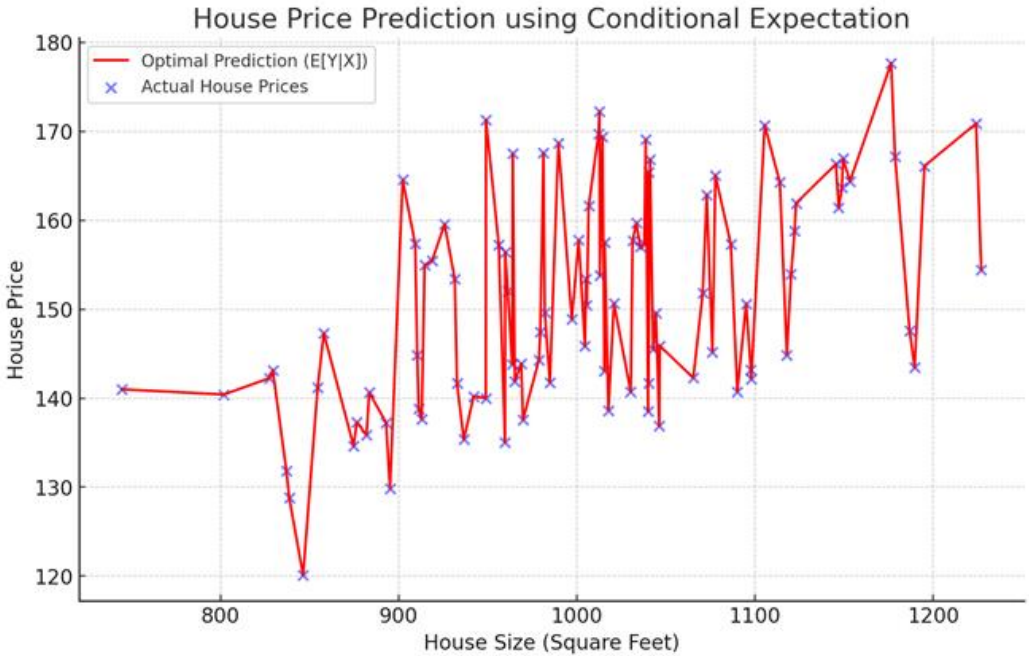
Polynomial Regression Equation (truncated):

$$y = 1.40e+00 - 2.75e+01x + 1.55e+02x^2 - 3.77e+02x^3 + 4.60e+02x^4 - 1.99e+02x^5 - \cdots$$

Mean Squared Error (MSE) Comparison:

- **Linear Regression MSE:** 0.186
- **Polynomial Regression (Degree 15) MSE:** 0.011

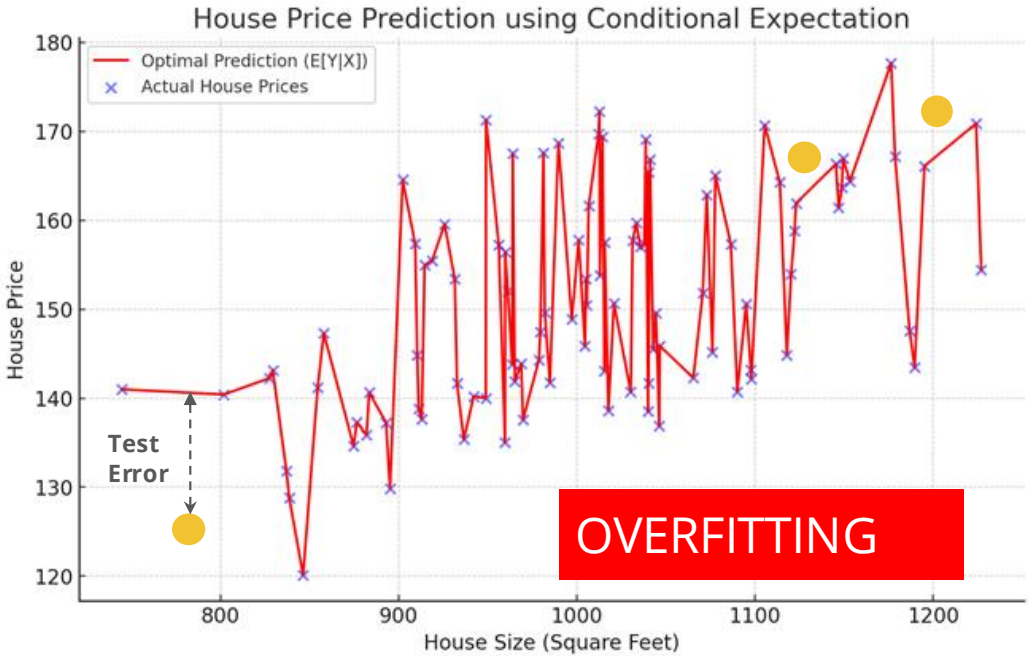
What is the MSE Train Loss?



Train Loss = 0

☆ how the model fits every trained data

What is the MSE Train/TEST Loss?



Train Loss = 0

Unseen (Test) Data

Test Loss > 0

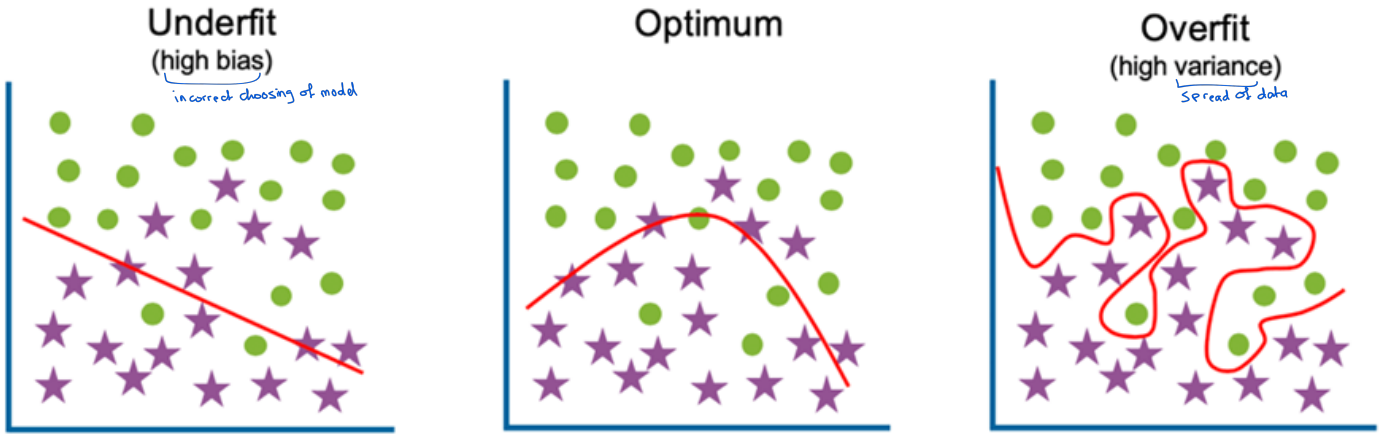
The model doesn't do well in tested data

Training Goal

The ultimate goal is **not merely to minimize training loss** but to build models that **generalize well** to **unseen data**.

Overfitting | Optimal | Underfit

In tests 1) after training the data, the error for trained and tested data is high
2) we can change the function to fix it like the function is a squared function when the model only learnt linear functions (polynomial)



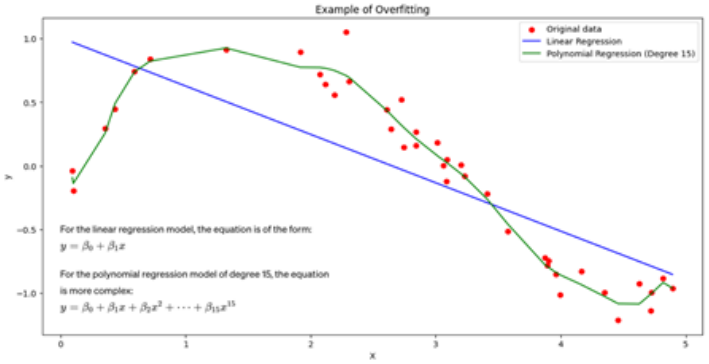
Tradeoff Bias / Variance

Understanding Bias vs. Variance

الانحياز والتباين

- **Objective**: Dissect the trade-offs between bias and variance in statistical learning models.
- **Definitions**:
  - **Bias**: The error due to overly simplistic assumptions in the learning algorithm. High bias can cause the model to miss relevant relations between features and target outputs (underfitting).
  - **Variance**: The error from sensitivity to small fluctuations in the training set. High variance can cause model to capture random noise in the training data (overfitting).
- **Key Concept**:
  - **Bias-Variance Tradeoff**: Balancing the complexity of the model to minimize both errors and achieve good generalization.
- **Illustration**:
  - High Bias: Assumes a linear relationship in a non-linear problem.
  - High Variance: Fits noise in the data as if it were a real pattern.
- **Mathematical Perspective**:
  - Expected prediction error for a given point x can be decomposed as:

$$\text{Error}(x) = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$



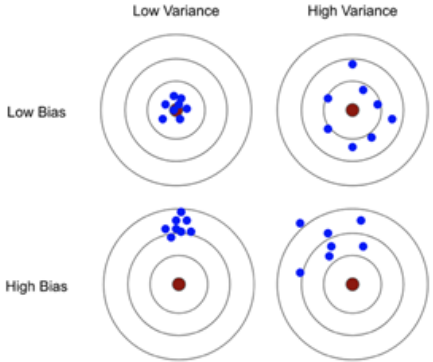
Simplifying the Bias-Variance Tradeoff

$$\text{Error}(x) = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

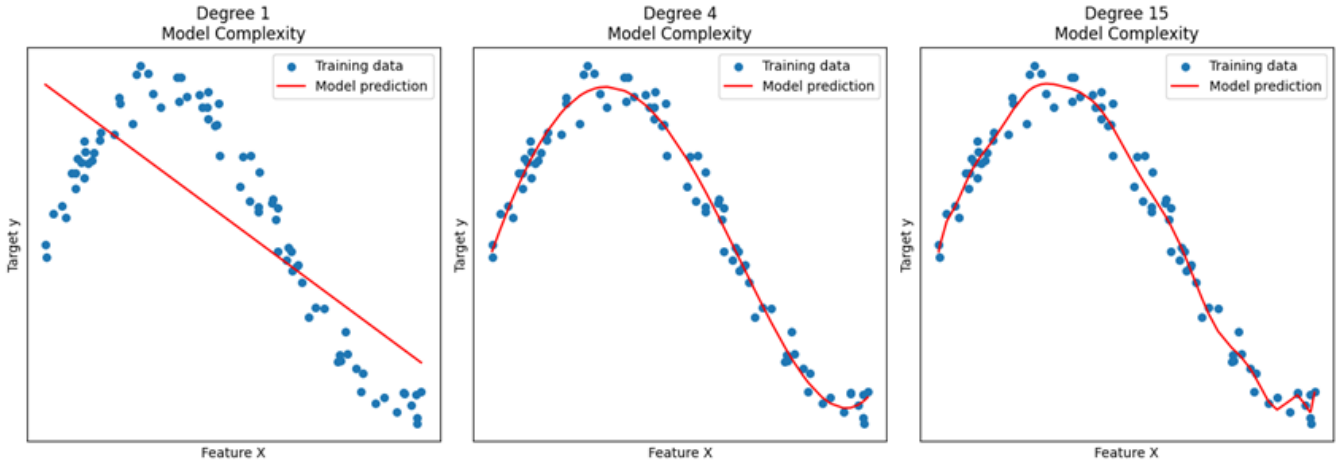
- **Understanding Prediction Error**:
  - Imagine you're trying to hit a target with arrows. Your goal is to be as close to the bullseye (the true value) as possible with each shot (prediction).
- **Components of Prediction Error**:
  - **Bias**: Suppose you consistently hit the target to the left of the bullseye. The average distance of your shots from the bullseye represents **Bias**. High bias means you're not even aiming correctly; your model is missing the mark because it's too simplistic.
  - **Variance**: Now, imagine your shots are spread out, sometimes to the left, sometimes to the right, above, and below the bullseye. The spread of your shots represents **Variance**. High variance means your model is sensitive to the training data and shoots all over the place.
  - **Irreducible Error**: There's always a bit of wind or some unseen bump in the ground that affects where your arrow lands, no matter how good your bow and your aim are. This is the **Irreducible Error**, which comes from randomness or unknown factors in the problem itself that we cannot predict or reduce.

- **Bias-Variance Tradeoff**: Balancing the complexity of the model to minimize both errors and achieve good generalization.
- **Illustration**:
  - High Bias: Assumes a linear relationship in a non-linear problem.
  - High Variance: Fits noise in the data as if it were a real pattern.
- **Mathematical Perspective**:
  - Expected prediction error for a given point x can be decomposed as:

$$\text{Error}(x) = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$



Understanding Bias vs. Variance

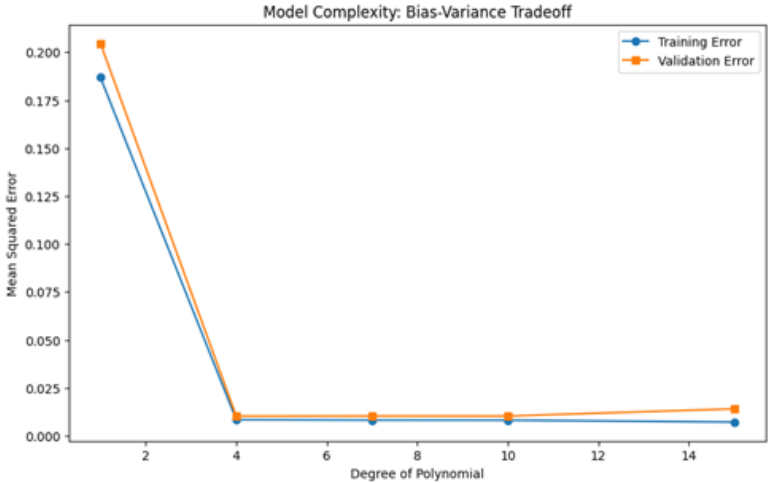


Understanding Bias vs. Variance

- **Training Error** decreases as the model complexity increases, which shows that the model is getting better at fitting the training data. However, this does not necessarily

mean it will perform well on unseen data.

- **Validation Error** initially decreases as the model becomes more capable of capturing the underlying pattern in the data but starts to increase at higher degrees of polynomial, indicating overfitting. This increase in validation error is due to the model's high variance, capturing noise in the training data as meaningful patterns, which do not generalize to new data.

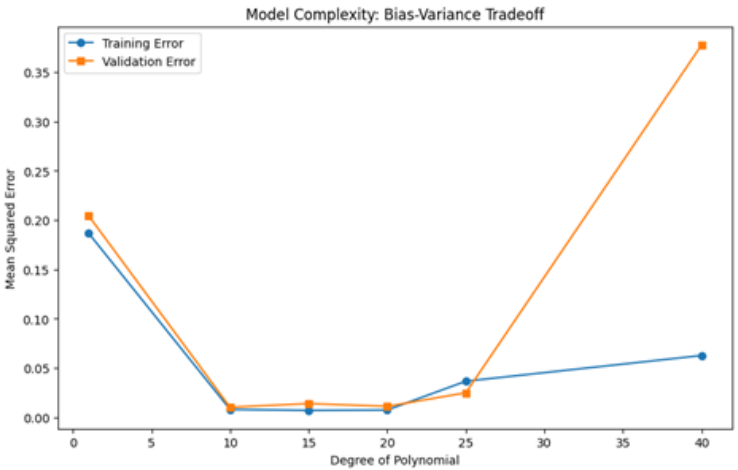


Understanding Bias vs. Variance

- **Training Error** decreases as the model complexity increases, which shows that the model is getting better at fitting the training data. However, this does not necessarily

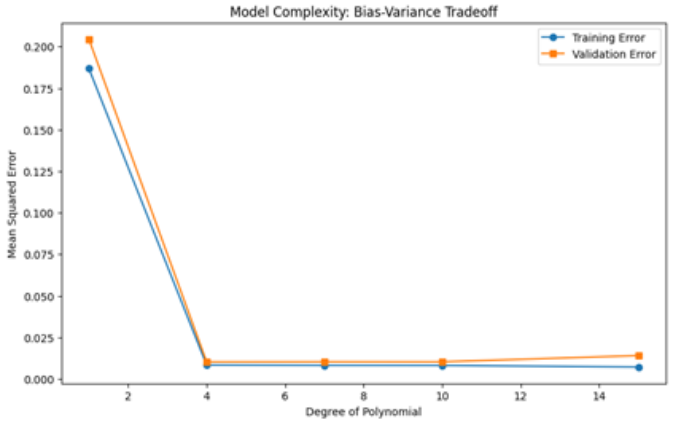
mean it will perform well on unseen data.

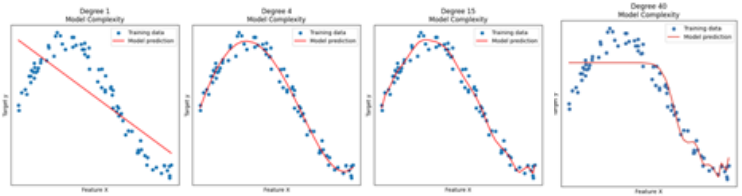
- **Validation Error** initially decreases as the model becomes more capable of capturing the underlying pattern in the data but starts to increase at higher degrees of polynomial, indicating overfitting. This increase in validation error is due to the model's high variance, capturing noise in the training data as meaningful patterns, which do not generalize to new data.



Understanding Bias vs. Variance

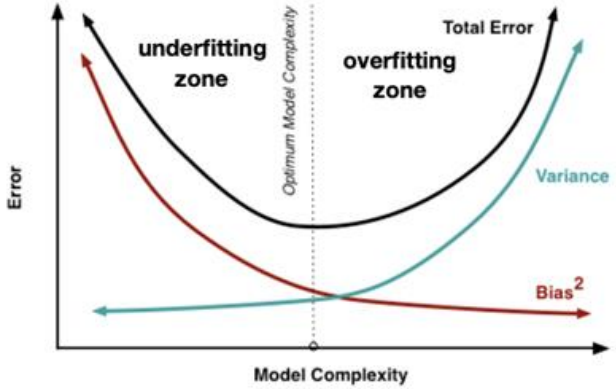
- **Bias**: At lower degrees of polynomial (e.g., 1), both training and validation errors are high because the model is too simple to capture the underlying pattern in the data (underfitting). This is indicative of high bias, where the model's assumptions are too far from the true relationship.
- **Variance**: At higher degrees of polynomial (e.g., 15), the training error is very low because the model fits the training data too closely, including noise. However, the validation error is high because the model fails to generalize to new data. This is indicative of high variance, where the model is too sensitive to small fluctuations in the training data.
- **Bias-Variance Tradeoff**: The optimal model complexity (in this case, around degree 4 to 7) achieves a balance between bias and variance. It is complex enough to capture the underlying pattern in the data (low bias) but not so complex that it fits the noise in the training data (controlled variance). This balance results in the lowest validation error, indicating the best generalization to unseen data.





How to Split Mismatched Distribution Data

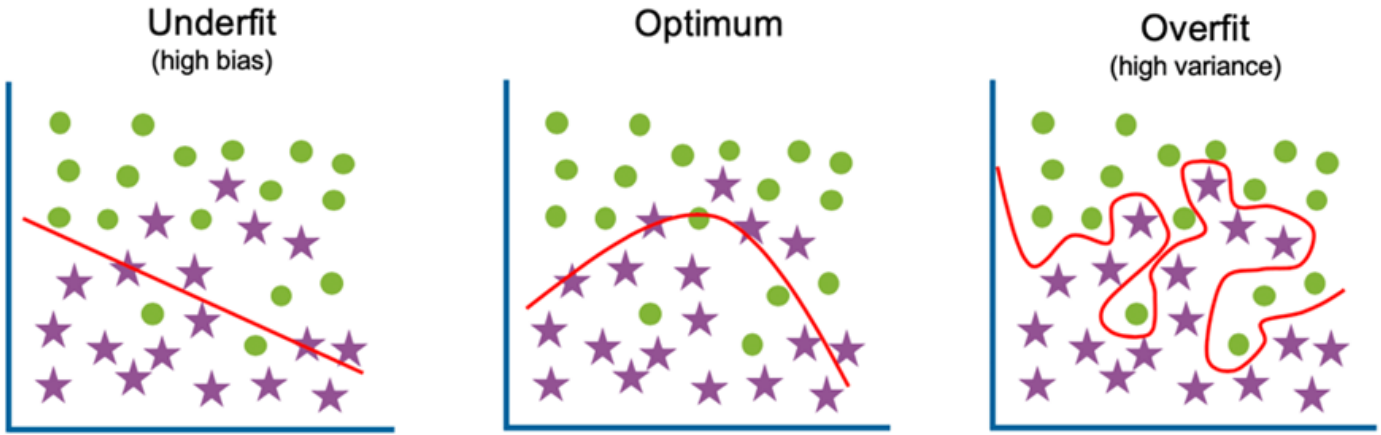
To build a good model, we need to find a good balance between bias and variance such that it minimizes the total error.



$$Err(x) = \left(E[\hat{f}(x)] - f(x)\right)^2 + E\left[\left(\hat{f}(x) - E[\hat{f}(x)]\right)^2\right] + \sigma_e^2$$

$$Err(x) = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

Overfitting | Optimal | Underfit



Bias and Variance Cases





Train Error	1%	15%	15%	0.5%
Test Error	11%	16%	30%	1%
Result	High Variance	High Bias	High Bias High Variance	Low Bias Low Variance
Fitting Type	Overfitting	Underfitting	Overfitting and Underfitting	Good Fitting

Error Decomposition in Predictive Modeling

Equation Breakdown

- **Error Decomposition Formula**:

$$E(Y - \hat{Y})^2 = E[\hat{g}(X) + \epsilon - g(X)]^2$$
$$= [\hat{g}(X) - g(X)]^2 + \mathrm{Var}(\epsilon)$$

Components Explained

1. **Total Prediction Error**:
   - Represents the expected value of the squared difference between actual and predicted values of Y.
2. **Reducible Error**:
   - $[\hat{g}(X) - g(X)]^2$: Squared difference between the estimated model $\hat{g}$ and the true model g.
   - Reflects the part of the error that can be reduced by improving the model estimation.
3. **Irreducible Error**:
   - $\mathrm{Var}(\epsilon)$: Variance of the error term ϵ.
   - Inherent uncertainty in the prediction process, unaffected by the model.

Understanding Reducible and Irreducible Errors

Reducible Error

- **Definition**: The part of the prediction error that can be decreased by improving the model's accuracy. It stems from the difference between the true model (g) and the estimated model ($\hat{g}$).
- **Formula**: $[g(X) - \hat{g}(X)]^2$
- **Characteristics**: Can be minimized through better model selection, more data, or improved learning techniques.

Irreducible Error

- **Definition**: The error inherent in the prediction process, unaffected by the model used. It arises from randomness or unknown variables that cannot be modeled or predicted.
- **Formula**: $\mathrm{Var}(\epsilon)$
- **Characteristics**: Sets the lower bound for the prediction error, representing noise in the data that cannot be eliminated.